Actividad 3

Programación Orientada a Objetos 2026-2S

Universidad Nacional de Colombia

Estudiante: Carlos Julio Nieto Morales

Docente: Walter Hugo Arboleda Mazo

Octubre 2026

Ejercicio 8.2 - Componentes Swing: Notas

Enunciado. Programa con interfaz gráfica que solicita cinco notas de un estudiante y muestra, en la parte inferior de la ventana, el promedio, la desviación estándar, la mayor nota y la menor nota.

Parte propuesta (resuelta). (1) Generar mensajes de alerta cuando se ingresen datos que no sean numéricos. (2) Es obligatorio ingresar las cinco notas: se muestra una alerta cuando una nota no ha sido ingresada. Adicionalmente se validó que cada nota esté entre 0,0 y 5,0 y se aceptan coma o punto como separador decimal.

Casos de uso

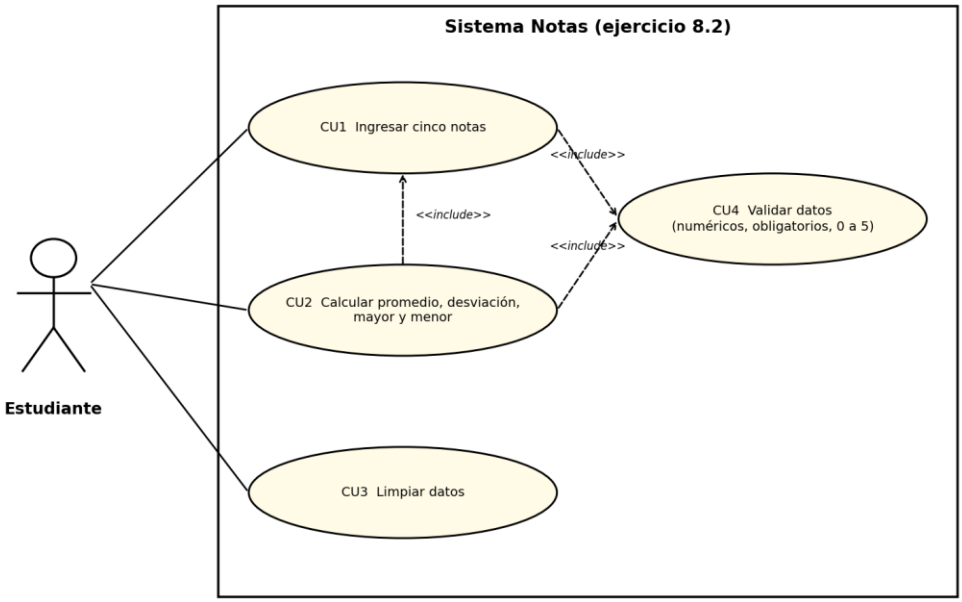


Figura 1. Diagrama de casos de uso del ejercicio 8.2.

Caso de uso	Actor	Descripción	Flujos alternos
CU1 Ingresar cinco notas	Estudiante	Escribe cada nota en su campo de texto.	Campo vacío: alerta de nota no ingresada (CU4).
CU2 Calcular métricas	Estudiante	Pulsa *Calcular*; el sistema muestra promedio, desviación estándar, mayor y menor.	Si CU4 falla, no se calcula y se enfoca el campo con error.
CU3 Limpiar datos	Estudiante	Pulsa *Limpiar*; se borran campos y resultados.	-
CU4 Validar datos	Sistema	Verifica que las cinco notas existan, sean numéricas y estén entre 0 y 5.	Muestra un *JOptionPane* de error y deja el foco en el campo.

Diagrama de clases

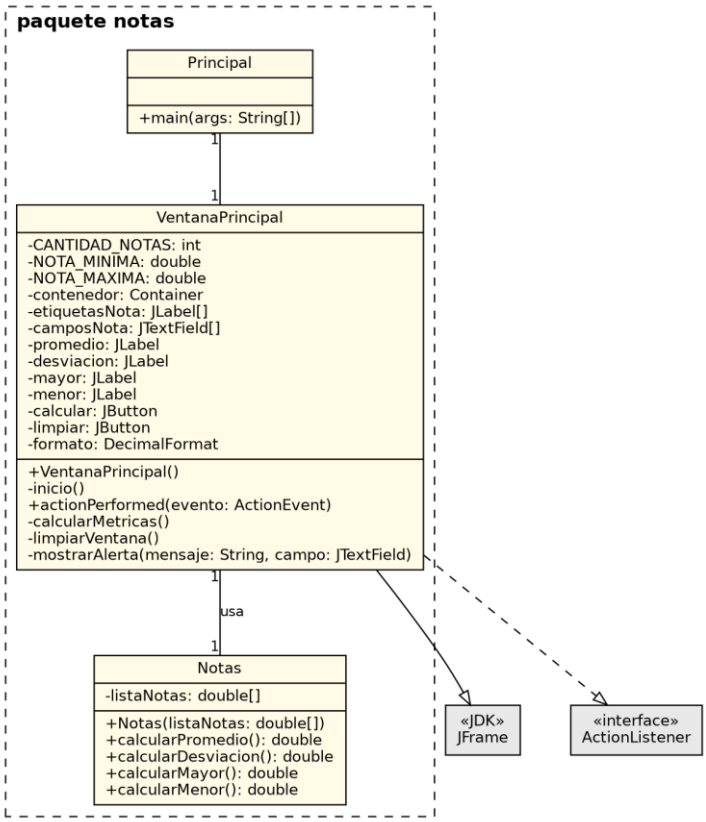


Figura 2. Diagrama de clases del ejercicio 8.2 (paquete notas).

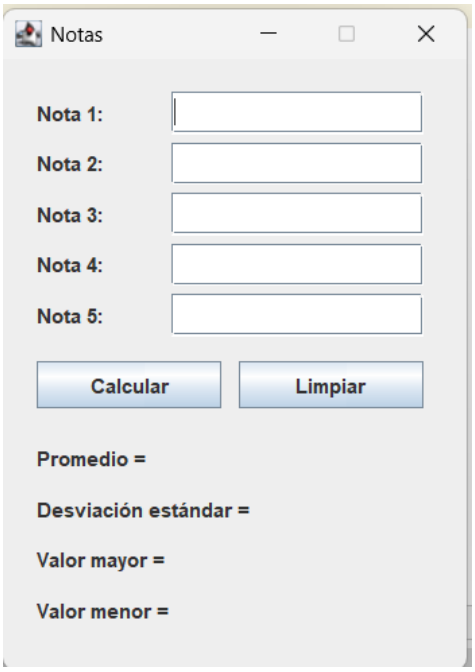
El punto de entrada es Principal (método *main*), asociada 1 a 1 con VentanaPrincipal, que extiende *JFrame* e implementa *ActionListener*: contiene los componentes gráficos (etiquetas, campos de texto y botones) y gestiona los eventos en *actionPerformed*. VentanaPrincipal usa una instancia de Notas, clase de dominio con un arreglo de notas y los métodos para calcular promedio, desviación estándar, mayor y menor. Se separa así la lógica de la interfaz.

Diagrama de objetos

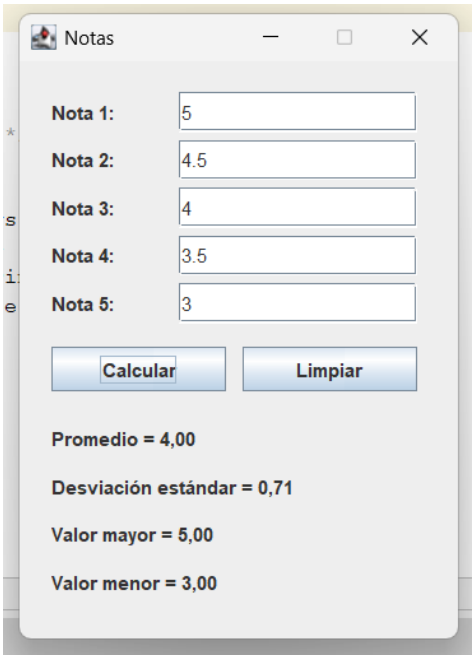


Figura 3. Diagrama de objetos del ejercicio 8.2 (ejecución con las notas 5, 4.5, 4, 3.5 y 3).

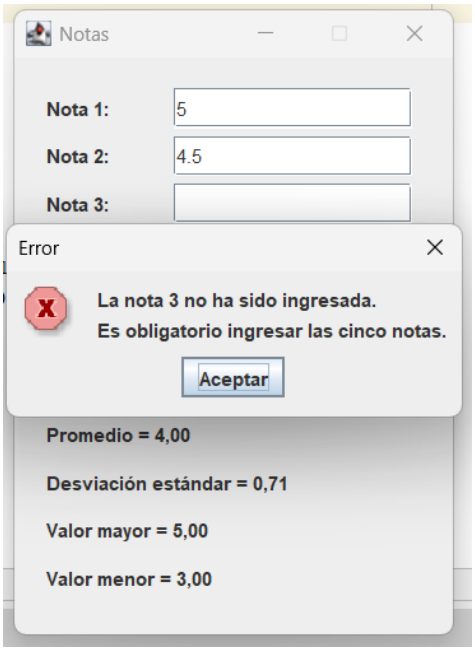
Interfaz de usuario y pruebas



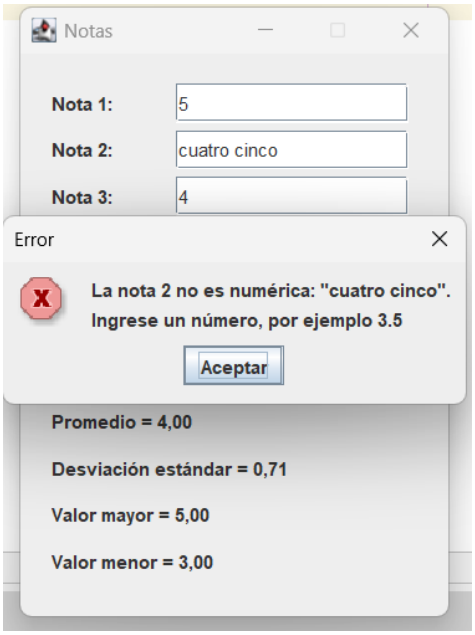
a) Ventana inicial



b) Resultado con 5, 4.5, 4, 3.5 y 3



c) Alerta: nota no ingresada



d) Alerta: dato no numérico

Figura 4. Ejecución del programa del ejercicio 8.2, incluida la parte propuesta.

Entrada	Promedio	Desv. estándar	Mayor	Menor
5; 4,5; 4; 3,5; 3	4,00	0,71	5,00	3,00

Nota: la desviación estándar se calcula como poblacional (se divide entre n = 5). Con la fórmula muestral (n - 1) el resultado sería 0,79. Los valores de la figura 8.6 del libro (promedio 3,00 y desviación 1,22) no corresponden a las notas que aparecen en esa figura.

Ejercicio 8.3 - Gestión de eventos: Figuras

Enunciado. Programa con una ventana principal con un botón por figura geométrica (cilindro, esfera y pirámide). Cada botón abre una ventana donde se ingresan las dimensiones y, al pulsar *Calcular*, se muestran el volumen y la superficie. Los eventos de los botones se gestionan con *ActionListener*.

Parte propuesta (resuelta):

(1) Agregar el cálculo de volumen y superficie de otras figuras: se agregaron el cubo y el prisma rectangular.

(2) Agregar a cada figura su imagen correspondiente: la clase *ImagenFigura* dibuja cada figura con Java2D.

Figura	Datos	Volumen	Superficie
Cilindro	radio r, altura h	$pi \cdot r^2 \cdot h$	$2 \cdot pi \cdot r \cdot (r + h)$
Esfera	radio r	$4/3 \cdot pi \cdot r^3$	$4 \cdot pi \cdot r^2$
Pirámide (base cuadrada)	base b, altura h, apotema a	$b^2 \cdot h / 3$	$b^2 + 2 \cdot b \cdot a$
Cubo (nuevo)	lado l	l^3	$6 \cdot l^2$
Prisma rectangular (nuevo)	largo, ancho, alto	largo · ancho · alto	2 · (largo·ancho + largo·alto + ancho·alto)

Casos de uso

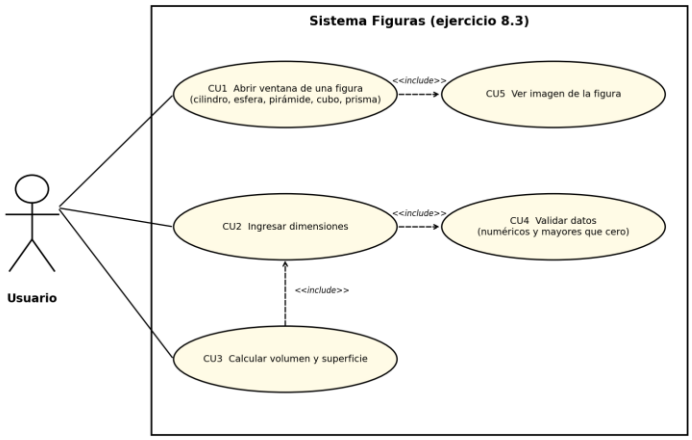


Figura 5. Diagrama de casos de uso del ejercicio 8.3.

Caso de uso	Actor	Descripción	Flujos alternos
CU1 Abrir ventana de una figura	Usuario	Pulsa el botón de la figura en la ventana principal y se abre su ventana.	-
CU2 Ingresar dimensiones	Usuario	Escribe radio, altura, base, apotema, lado, etc., según la figura.	Campo vacío o no numérico: alerta (CU4).
CU3 Calcular volumen y superficie	Usuario	Pulsa *Calcular*; se muestran volumen (cm3) y superficie (cm2).	Si CU4 falla, no se calcula.
CU4 Validar datos	Sistema	Comprueba que los datos existan, sean numéricos y mayores que cero.	Muestra *JOptionPane* de error.
CU5 Ver imagen de la figura	Usuario	Cada ventana muestra el dibujo de su figura.	-

Diagrama de clases

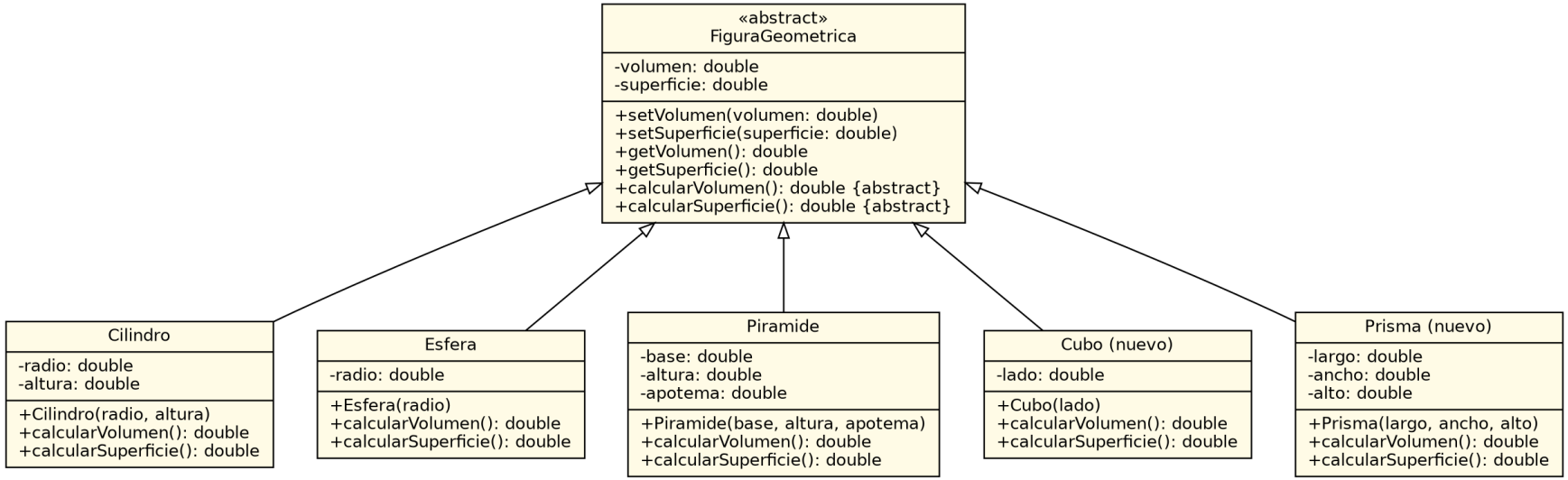


Figura 6. Diagrama de clases del ejercicio 8.3 - jerarquía de figuras (dominio).

Diagrama de clases (continuación)

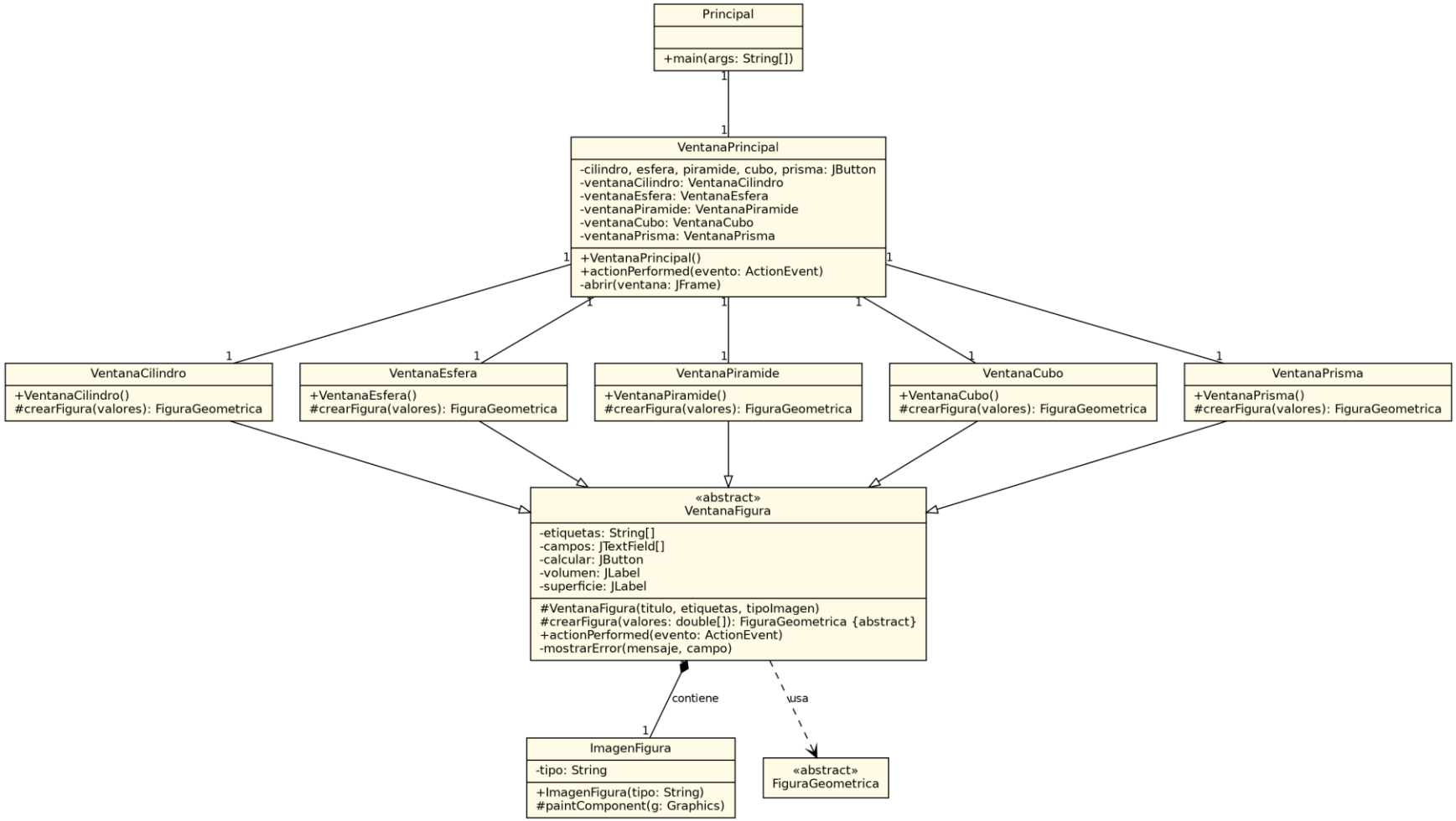


Figura 7. Diagrama de clases del ejercicio 8.3 - capa de interfaz gráfica.

Explicación del diagrama de clases

El paquete figuras se organiza en dos partes.

Dominio: FiguraGeometrica es una clase abstracta con los atributos volumen y superficie, sus *getters/setters* y los métodos abstractos *calcularVolumen* y *calcularSuperficie*. Cilindro, Esfera, Pirámide, Cubo y Prisma heredan de ella y los implementan (polimorfismo).

Interfaz: Principal (*main*) crea la VentanaPrincipal, que tiene cinco botones y una instancia de cada ventana de figura (asociación 1 a 1). Para no repetir código, las cinco ventanas heredan de la clase abstracta VentanaFigura, que construye los campos, valida, muestra los resultados e incluye una ImagenFigura; cada ventana concreta solo indica sus etiquetas y cómo crear su figura (*crearFigura*).

Diagrama de objetos

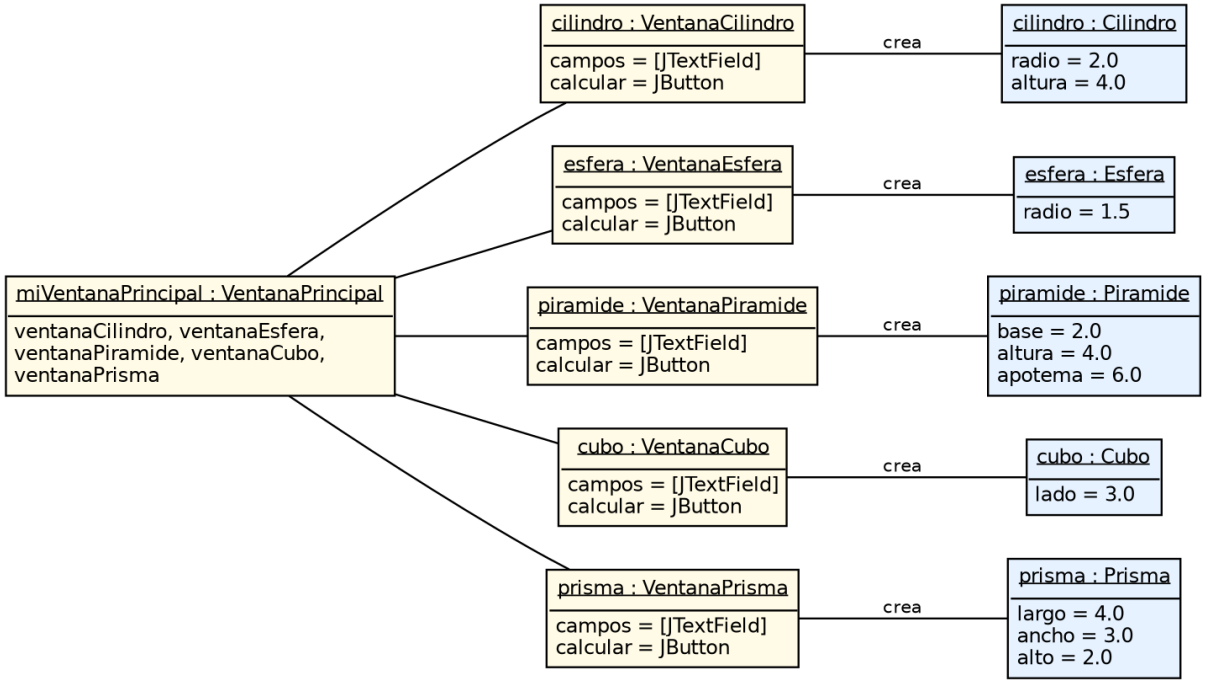
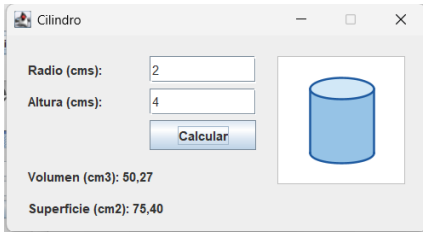


Figura 8. Diagrama de objetos del ejercicio 8.3 (ejecución con los datos de la figura 9).

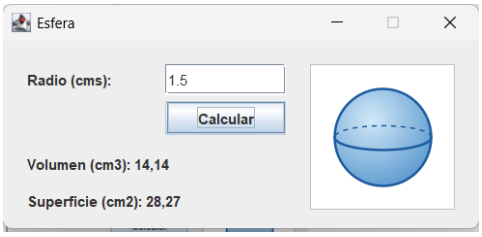
Interfaz de usuario y pruebas



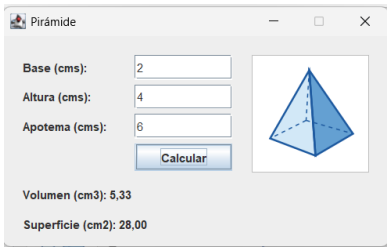
a) Ventana principal



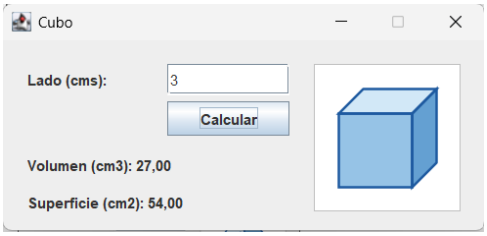
b) Cilindro



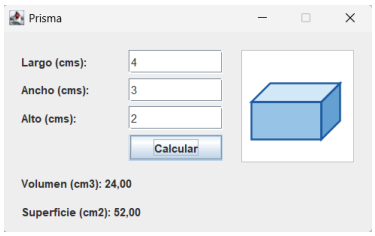
c) Esfera



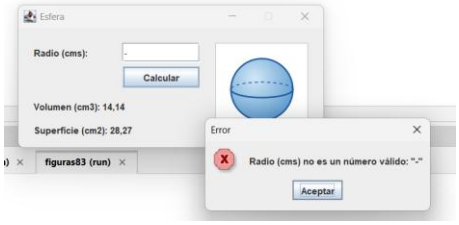
d) Pirámide



e) Cubo (nuevo)



f) Prisma (nuevo)



g) Alerta de validación

Figura 9. Ejecución del programa del ejercicio 8.3, incluida la parte propuesta.

Figura	Datos de prueba	Volumen (cm3)	Superficie (cm2)
Cilindro	r = 2; h = 4	50,27	75,40
Esfera	r = 1,5	14,14	28,27
Pirámide	b = 2; h = 4; a = 6	5,33	28,00
Cubo	l = 3	27,00	54,00
Prisma	4 x 3 x 2	24,00	52,00

Nota: en la figura 8.9 del libro el volumen del cilindro aparece como 60,37, pero con r = 2 y h = 4 el volumen es $pi \cdot 2^2 \cdot 4 = 50{,}27$, que es el valor que calcula este programa. La esfera da 14,137 (el libro muestra 14,13 ; por aproximacion aquí se redondea a 14,14).

Enlace GitHub:

https://github.com/CarlosJulioNietoMorales/POO-2026-02/tree/main/Actividades/Actividad%20%233

Código fuente del ejercicio 8.2 (paquete notas)

Principal.java

```
package notas;

import javax.swing.SwingUtilities;

/** Punto de entrada del ejercicio 8.2. */
public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            VentanaPrincipal miVentanaPrincipal = new VentanaPrincipal();
            miVentanaPrincipal.setVisible(true);
        });
    }
}
```

VentanaPrincipal.java

```
package notas;

import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;

/**
 * Ventana del ejercicio 8.2: solicita cinco notas y muestra promedio,
 * desviacion estandar, mayor y menor.
 * Parte propuesta: alertas cuando el dato no es numerico y cuando falta una nota.
 */
public class VentanaPrincipal extends JFrame implements ActionListener {

    private static final int CANTIDAD_NOTAS = 5;
    private static final double NOTA_MINIMA = 0.0;
    private static final double NOTA_MAXIMA = 5.0;

    private Container contenedor;
    private JLabel[] etiquetasNota;
    private JTextField[] camposNota;
    private JLabel promedio;
    private JLabel desviacion;
    private JLabel mayor;
    private JLabel menor;
    private JButton calcular;
    private JButton limpiar;
    private DecimalFormat formato;

    public VentanaPrincipal() {
        DecimalFormatSymbols simbolos = new DecimalFormatSymbols();
        simbolos.setDecimalSeparator(',');
        formato = new DecimalFormat("0.00", simbolos);
        inicio();
    }

    private void inicio() {
        contenedor = getContentPane();
        contenedor.setLayout(null);

        etiquetasNota = new JLabel[CANTIDAD_NOTAS];
        camposNota = new JTextField[CANTIDAD_NOTAS];
        for (int i = 0; i < CANTIDAD_NOTAS; i++) {
            etiquetasNota[i] = new JLabel("Nota " + (i + 1) + ":");
            etiquetasNota[i].setBounds(20, 20 + 30 * i, 70, 25);
            camposNota[i] = new JTextField();
            camposNota[i].setBounds(100, 20 + 30 * i, 150, 25);
            contenedor.add(etiquetasNota[i]);
            contenedor.add(camposNota[i]);
        }

        calcular = new JButton("Calcular");
        calcular.setBounds(20, 180, 110, 28);
        calcular.addActionListener(this);
        limpiar = new JButton("Limpiar");
        limpiar.setBounds(140, 180, 110, 28);
        limpiar.addActionListener(this);
        contenedor.add(calcular);
```

```
        contenedor.add(limpiar);

        promedio = new JLabel("Promedio = ");
        promedio.setBounds(20, 225, 230, 25);
        desviacion = new JLabel("Desviación estándar = ");
        desviacion.setBounds(20, 255, 230, 25);
        mayor = new JLabel("Valor mayor = ");
        mayor.setBounds(20, 285, 230, 25);
        menor = new JLabel("Valor menor = ");
        menor.setBounds(20, 315, 230, 25);
        contenedor.add(promedio);
        contenedor.add(desviacion);
        contenedor.add(mayor);
        contenedor.add(menor);

        setTitle("Notas");
        setSize(290, 400);
        setResizable(false);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        if (evento.getSource() == calcular) {
            calcularMetricas();
        } else if (evento.getSource() == limpiar) {
            limpiarVentana();
        }
    }

    private void calcularMetricas() {
        double[] valores = new double[CANTIDAD_NOTAS];
        for (int i = 0; i < CANTIDAD_NOTAS; i++) {
            String texto = camposNota[i].getText().trim();

            // Propuesto 2: las cinco notas son obligatorias
            if (texto.isEmpty()) {
                mostrarAlerta("La nota " + (i + 1) + " no ha sido ingresada.\n"
                        + "Es obligatorio ingresar las cinco notas.", camposNota[i]);
                return;
            }

            // Propuesto 1: solo se aceptan datos numericos (se admite coma o punto decimal)
            double valor;
            try {
                valor = Double.parseDouble(texto.replace(',', '.'));
            } catch (NumberFormatException ex) {
                mostrarAlerta("La nota " + (i + 1) + " no es numérica: \"" + texto + "\".\n"
                        + "Ingrese un número, por ejemplo 3.5", camposNota[i]);
                return;
            }

            // Validacion adicional: las notas estan entre 0 y 5 (tambien descarta NaN)
            if (!(valor >= NOTA_MINIMA && valor <= NOTA_MAXIMA)) {
                mostrarAlerta("La nota " + (i + 1) + " debe estar entre "
                        + NOTA_MINIMA + " y " + NOTA_MAXIMA + ".", camposNota[i]);
                return;
            }
            valores[i] = valor;
        }

        Notas notas = new Notas(valores);
        promedio.setText("Promedio = " + formato.format(notas.calcularPromedio()));
        desviacion.setText("Desviación estándar = " + formato.format(notas.calcularDesviacion()));
        mayor.setText("Valor mayor = " + formato.format(notas.calcularMayor()));
        menor.setText("Valor menor = " + formato.format(notas.calcularMenor()));
    }

    private void limpiarVentana() {
        for (JTextField campo : camposNota) {
            campo.setText("");
        }
        promedio.setText("Promedio = ");
        desviacion.setText("Desviación estándar = ");
        mayor.setText("Valor mayor = ");
        menor.setText("Valor menor = ");
        camposNota[0].requestFocus();
    }

    private void mostrarAlerta(String mensaje, JTextField campo) {
        JOptionPane.showMessageDialog(this, mensaje, "Error", JOptionPane.ERROR_MESSAGE);
        campo.requestFocus();
        campo.selectAll();
    }
}
```

Notas.java

```
package notas;
```

```java
/**
 * Clase de dominio: guarda las notas de un estudiante y calcula
 * las metricas pedidas (promedio, desviacion estandar, mayor y menor).
 */
public class Notas {

    private double[] listaNotas;

    public Notas(double[] listaNotas) {
        if (listaNotas == null || listaNotas.length == 0) {
            throw new IllegalArgumentException("Debe existir al menos una nota");
        }
        this.listaNotas = listaNotas.clone();
    }

    public double calcularPromedio() {
        double suma = 0;
        for (double nota : listaNotas) {
            suma += nota;
        }
        return suma / listaNotas.length;
    }

    /** Desviacion estandar poblacional: raiz de la media de los cuadrados de las diferencias. */
    public double calcularDesviacion() {
        double promedio = calcularPromedio();
        double suma = 0;
        for (double nota : listaNotas) {
            suma += Math.pow(nota - promedio, 2);
        }
        return Math.sqrt(suma / listaNotas.length);
    }

    public double calcularMayor() {
        double mayor = listaNotas[0];
        for (double nota : listaNotas) {
            if (nota > mayor) {
                mayor = nota;
            }
        }
        return mayor;
    }

    public double calcularMenor() {
        double menor = listaNotas[0];
        for (double nota : listaNotas) {
            if (nota < menor) {
                menor = nota;
            }
        }
        return menor;
    }
}
```

Código fuente del ejercicio 8.3 (paquete figuras)

Principal.java

```
package figuras;

import javax.swing.SwingUtilities;

/** Punto de entrada del ejercicio 8.3. */
public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            VentanaPrincipal miVentanaPrincipal = new VentanaPrincipal();
            miVentanaPrincipal.setVisible(true);
        });
    }
}
```

VentanaPrincipal.java

```
package figuras;

import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;

/** Ventana principal: un boton por figura que abre su ventana de calculo. */
public class VentanaPrincipal extends JFrame implements ActionListener {

    private Container contenedor;
    private JButton cilindro;
    private JButton esfera;
    private JButton piramide;
    private JButton cubo;
    private JButton prisma;
    private VentanaCilindro ventanaCilindro;
    private VentanaEsfera ventanaEsfera;
    private VentanaPiramide ventanaPiramide;
    private VentanaCubo ventanaCubo;
    private VentanaPrisma ventanaPrisma;

    public VentanaPrincipal() {
        inicio();
    }

    private void inicio() {
        contenedor = getContentPane();
        contenedor.setLayout(null);

        cilindro = crearBoton("Cilindro", 15);
        esfera = crearBoton("Esfera", 125);
        piramide = crearBoton("Pirámide", 235);
        cubo = crearBoton("Cubo", 345);
        prisma = crearBoton("Prisma", 455);

        ventanaCilindro = new VentanaCilindro();
        ventanaEsfera = new VentanaEsfera();
        ventanaPiramide = new VentanaPiramide();
        ventanaCubo = new VentanaCubo();
        ventanaPrisma = new VentanaPrisma();

        setTitle("Figuras");
        setSize(585, 120);
        setResizable(false);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    private JButton crearBoton(String texto, int x) {
        JButton boton = new JButton(texto);
        boton.setBounds(x, 25, 105, 28);
        boton.addActionListener(this);
        contenedor.add(boton);
        return boton;
    }

    @Override
    public void actionPerformed(ActionEvent evento) {
        Object origen = evento.getSource();
        if (origen == cilindro) {
            abrir(ventanaCilindro);
        } else if (origen == esfera) {
            abrir(ventanaEsfera);
```

```
        } else if (origen == piramide) {
            abrir(ventanaPiramide);
        } else if (origen == cubo) {
            abrir(ventanaCubo);
        } else if (origen == prisma) {
            abrir(ventanaPrisma);
        }
    }

    private void abrir(JFrame ventana) {
        ventana.setVisible(true);
        ventana.toFront();
    }
}
```

VentanaFigura.java

```
package figuras;

import java.awt.Container;
import java.awt.Dimension;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.text.DecimalFormat;
import java.text.DecimalFormatSymbols;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextField;

/**
 * Ventana generica de una figura: pide las dimensiones, valida los datos y muestra
 * volumen y superficie. Cada figura solo define sus etiquetas y como crear el objeto.
 */
public abstract class VentanaFigura extends JFrame implements ActionListener {

    private final String[] etiquetas;
    private final JTextField[] campos;
    private final JButton calcular;
    private final JLabel volumen;
    private final JLabel superficie;
    private final DecimalFormat formato;

    protected VentanaFigura(String titulo, String[] etiquetas, String tipoImagen) {
        super(titulo);
        this.etiquetas = etiquetas;
        DecimalFormatSymbols simbolos = new DecimalFormatSymbols();
        simbolos.setDecimalSeparator(',');
        formato = new DecimalFormat("0.00", simbolos);

        Container contenedor = getContentPane();
        contenedor.setLayout(null);

        campos = new JTextField[etiquetas.length];
        for (int i = 0; i < etiquetas.length; i++) {
            JLabel etiqueta = new JLabel(etiquetas[i]);
            etiqueta.setBounds(20, 20 + 30 * i, 110, 25);
            campos[i] = new JTextField();
            campos[i].setBounds(135, 20 + 30 * i, 100, 25);
            contenedor.add(etiqueta);
            contenedor.add(campos[i]);
        }

        int y = 20 + 30 * etiquetas.length;
        calcular = new JButton("Calcular");
        calcular.setBounds(135, y, 100, 28);
        calcular.addActionListener(this);
        contenedor.add(calcular);

        volumen = new JLabel("Volumen (cm3): ");
        volumen.setBounds(20, y + 40, 225, 25);
        superficie = new JLabel("Superficie (cm2): ");
        superficie.setBounds(20, y + 70, 225, 25);
        contenedor.add(volumen);
        contenedor.add(superficie);

        ImagenFigura imagen = new ImagenFigura(tipoImagen);
        imagen.setBounds(255, 20, 120, 120);
        contenedor.add(imagen);

        contenedor.setPreferredSize(new Dimension(395, Math.max(155, y + 105)));
        pack();
        setResizable(false);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(HIDE_ON_CLOSE);
    }

    /** Cada figura concreta crea su objeto con los valores ingresados (en orden). */
    protected abstract FiguraGeometrica crearFigura(double[] valores);
```

```
    @Override
    public void actionPerformed(ActionEvent evento) {
        double[] valores = new double[campos.length];
        for (int i = 0; i < campos.length; i++) {
            String nombre = etiquetas[i].replace(":", "");
            String texto = campos[i].getText().trim();
            if (texto.isEmpty()) {
                mostrarError("Falta ingresar: " + nombre, campos[i]);
                return;
            }
            double valor;
            try {
                valor = Double.parseDouble(texto.replace(',', '.'));
            } catch (NumberFormatException ex) {
                mostrarError(nombre + " no es un número válido: \"" + texto + "\"", campos[i]);
                return;
            }
            if (!(valor > 0) || Double.isInfinite(valor)) {
                mostrarError(nombre + " debe ser un número mayor que cero", campos[i]);
                return;
            }
            valores[i] = valor;
        }
        FiguraGeometrica figura = crearFigura(valores);
        volumen.setText("Volumen (cm3): " + formato.format(figura.calcularVolumen()));
        superficie.setText("Superficie (cm2): " + formato.format(figura.calcularSuperficie()));
    }

    private void mostrarError(String mensaje, JTextField campo) {
        JOptionPane.showMessageDialog(this, mensaje, "Error", JOptionPane.ERROR_MESSAGE);
        campo.requestFocus();
        campo.selectAll();
    }
}
```

VentanaCilindro.java

```
package figuras;

public class VentanaCilindro extends VentanaFigura {

    public VentanaCilindro() {
        super("Cilindro", new String[] {"Radio (cms):", "Altura (cms):"}, "Cilindro");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Cilindro(valores[0], valores[1]);
    }
}
```

VentanaEsfera.java

```
package figuras;

public class VentanaEsfera extends VentanaFigura {

    public VentanaEsfera() {
        super("Esfera", new String[] {"Radio (cms):"}, "Esfera");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Esfera(valores[0]);
    }
}
```

VentanaPiramide.java

```
package figuras;

public class VentanaPiramide extends VentanaFigura {

    public VentanaPiramide() {
        super("Pirámide", new String[] {"Base (cms):", "Altura (cms):", "Apotema (cms):"}, "Pirámide");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Piramide(valores[0], valores[1], valores[2]);
    }
}
```

VentanaCubo.java

```
package figuras;

public class VentanaCubo extends VentanaFigura {
```

```
    public VentanaCubo() {
        super("Cubo", new String[] {"Lado (cms):"}, "Cubo");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Cubo(valores[0]);
    }
}
```

VentanaPrisma.java

```
package figuras;

public class VentanaPrisma extends VentanaFigura {

    public VentanaPrisma() {
        super("Prisma", new String[] {"Largo (cms):", "Ancho (cms):", "Alto (cms):"}, "Prisma");
    }

    @Override
    protected FiguraGeometrica crearFigura(double[] valores) {
        return new Prisma(valores[0], valores[1], valores[2]);
    }
}
```

ImagenFigura.java

```
package figuras;

import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Polygon;
import java.awt.RenderingHints;
import javax.swing.BorderFactory;
import javax.swing.JPanel;

/** Panel que dibuja la imagen de cada figura con Java2D (parte propuesta: imagen por figura). */
public class ImagenFigura extends JPanel {

    private static final Color BORDE = new Color(30, 90, 160);
    private static final Color CLARO = new Color(210, 232, 247);
    private static final Color MEDIO = new Color(150, 195, 230);
    private static final Color OSCURO = new Color(100, 160, 210);

    private final String tipo;

    public ImagenFigura(String tipo) {
        this.tipo = tipo;
        setPreferredSize(new Dimension(120, 120));
        setBackground(Color.WHITE);
        setBorder(BorderFactory.createLineBorder(Color.LIGHT_GRAY));
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setStroke(new BasicStroke(2f));
        switch (tipo) {
            case "Cilindro": dibujarCilindro(g2); break;
            case "Esfera": dibujarEsfera(g2); break;
            case "Pirámide": dibujarPiramide(g2); break;
            case "Cubo": dibujarCaja(g2, 20, 40, 60, 60, 20); break;
            case "Prisma": dibujarCaja(g2, 10, 55, 75, 40, 20); break;
            default: break;
        }
        g2.dispose();
    }

    private void dibujarCilindro(Graphics2D g2) {
        g2.setColor(MEDIO);
        g2.fillRect(30, 30, 60, 60);
        g2.fillOval(30, 80, 60, 20);
        g2.setColor(BORDE);
        g2.drawArc(30, 80, 60, 20, 180, 180);
        g2.drawLine(30, 30, 30, 90);
        g2.drawLine(90, 30, 90, 90);
        g2.setColor(CLARO);
        g2.fillOval(30, 20, 60, 20);
        g2.setColor(BORDE);
        g2.drawOval(30, 20, 60, 20);
    }

    private void dibujarEsfera(Graphics2D g2) {
```

```
        g2.setPaint(new java.awt.RadialGradientPaint(50, 45, 55,
                new float[] {0f, 1f}, new Color[] {CLARO, OSCURO}));
        g2.fillOval(20, 20, 80, 80);
        g2.setColor(BORDE);
        g2.drawOval(20, 20, 80, 80);
        g2.setStroke(new BasicStroke(1.2f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER,
                10f, new float[] {4f, 4f}, 0f));
        g2.drawArc(20, 50, 80, 20, 0, 180);
        g2.setStroke(new BasicStroke(2f));
        g2.drawArc(20, 50, 80, 20, 180, 180);
    }

    private void dibujarPiramide(Graphics2D g2) {
        Polygon izquierda = new Polygon(new int[] {58, 18, 65}, new int[] {15, 85, 102}, 3);
        Polygon derecha = new Polygon(new int[] {58, 65, 103}, new int[] {15, 102, 80}, 3);
        g2.setColor(CLARO);
        g2.fillPolygon(izquierda);
        g2.setColor(OSCURO);
        g2.fillPolygon(derecha);
        g2.setColor(BORDE);
        g2.drawPolygon(izquierda);
        g2.drawPolygon(derecha);
        g2.setStroke(new BasicStroke(1.2f, BasicStroke.CAP_BUTT, BasicStroke.JOIN_MITER,
                10f, new float[] {4f, 4f}, 0f));
        g2.drawLine(58, 15, 55, 66);
        g2.drawLine(55, 66, 18, 85);
        g2.drawLine(55, 66, 103, 80);
    }

    /** Dibuja un paralelepipedo: cara frontal (x, y, ancho, alto) y profundidad d. */
    private void dibujarCaja(Graphics2D g2, int x, int y, int w, int h, int d) {
        Polygon frente = new Polygon(new int[] {x, x + w, x + w, x}, new int[] {y, y, y + h, y + h}, 4);
        Polygon arriba = new Polygon(new int[] {x, x + w, x + w + d, x + d},
                new int[] {y, y, y - d, y - d}, 4);
        Polygon lado = new Polygon(new int[] {x + w, x + w + d, x + w + d, x + w},
                new int[] {y, y - d, y + h - d, y + h}, 4);
        g2.setColor(MEDIO);
        g2.fillPolygon(frente);
        g2.setColor(CLARO);
        g2.fillPolygon(arriba);
        g2.setColor(OSCURO);
        g2.fillPolygon(lado);
        g2.setColor(BORDE);
        g2.drawPolygon(frente);
        g2.drawPolygon(arriba);
        g2.drawPolygon(lado);
    }
}
```

FiguraGeometrica.java

```
package figuras;

/** Clase base de todas las figuras: guarda volumen y superficie calculados. */
public abstract class FiguraGeometrica {

    private double volumen;
    private double superficie;

    public void setVolumen(double volumen) {
        this.volumen = volumen;
    }

    public void setSuperficie(double superficie) {
        this.superficie = superficie;
    }

    public double getVolumen() {
        return volumen;
    }

    public double getSuperficie() {
        return superficie;
    }

    public abstract double calcularVolumen();

    public abstract double calcularSuperficie();
}
```

Cilindro.java

```
package figuras;

public class Cilindro extends FiguraGeometrica {

    private double radio;
    private double altura;

    public Cilindro(double radio, double altura) {
```

```
        this.radio = radio;
        this.altura = altura;
    }

    @Override
    public double calcularVolumen() {
        double v = Math.PI * radio * radio * altura;
        setVolumen(v);
        return v;
    }

    @Override
    public double calcularSuperficie() {
        double s = 2 * Math.PI * radio * (radio + altura);
        setSuperficie(s);
        return s;
    }
}
```

Esfera.java

```
package figuras;

public class Esfera extends FiguraGeometrica {

    private double radio;

    public Esfera(double radio) {
        this.radio = radio;
    }

    @Override
    public double calcularVolumen() {
        double v = 4.0 / 3.0 * Math.PI * Math.pow(radio, 3);
        setVolumen(v);
        return v;
    }

    @Override
    public double calcularSuperficie() {
        double s = 4 * Math.PI * radio * radio;
        setSuperficie(s);
        return s;
    }
}
```

Piramide.java

```
package figuras;

/** Piramide de base cuadrada: base = lado de la base, apotema = altura de una cara lateral. */
public class Piramide extends FiguraGeometrica {

    private double base;
    private double altura;
    private double apotema;

    public Piramide(double base, double altura, double apotema) {
        this.base = base;
        this.altura = altura;
        this.apotema = apotema;
    }

    @Override
    public double calcularVolumen() {
        double v = base * base * altura / 3.0;
        setVolumen(v);
        return v;
    }

    @Override
    public double calcularSuperficie() {
        double s = base * base + 2 * base * apotema;
        setSuperficie(s);
        return s;
    }
}
```

Cubo.java

```
package figuras;

/** Figura nueva (parte propuesta). */
public class Cubo extends FiguraGeometrica {

    private double lado;

    public Cubo(double lado) {
        this.lado = lado;
```

```
    }

    @Override
    public double calcularVolumen() {
        double v = Math.pow(lado, 3);
        setVolumen(v);
        return v;
    }

    @Override
    public double calcularSuperficie() {
        double s = 6 * lado * lado;
        setSuperficie(s);
        return s;
    }
}
```

Prisma.java

```
package figuras;

/** Prisma rectangular (figura nueva, parte propuesta). */
public class Prisma extends FiguraGeometrica {

    private double largo;
    private double ancho;
    private double alto;

    public Prisma(double largo, double ancho, double alto) {
        this.largo = largo;
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    public double calcularVolumen() {
        double v = largo * ancho * alto;
        setVolumen(v);
        return v;
    }

    @Override
    public double calcularSuperficie() {
        double s = 2 * (largo * ancho + largo * alto + ancho * alto);
        setSuperficie(s);
        return s;
    }
}
```